Practical – 12

Aim – 1 : Word Frequency

Split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator.

Code :

Main.java

```java
import java.util.*;

public class WordFrequency {
    public static void main(String[] args) {

        String paragraph = "java is easy and java is powerful and easy";

        // Split paragraph into words
        String[] words = paragraph.toLowerCase().split("\\s+");

        // Count frequency using HashMap
        HashMap<String, Integer> map = new HashMap<>();

        for (String word : words) {
            map.put(word, map.getOrDefault(word, 0) + 1);
        }

        // Sort words by frequency (highest first)
        List<Map.Entry<String, Integer>> list =
                new ArrayList<>(map.entrySet());

        list.sort((a, b) -> {
            int result = b.getValue().compareTo(a.getValue());

            // If frequency is same, sort alphabetically
            if (result == 0) {
                return a.getKey().compareTo(b.getKey());
            }

            return result;
```

Terminal

```java
public class WordFrequency {
    public static void main(String[] args) {

        for (String word : words) {
            map.put(word, map.getOrDefault(word, 0) + 1);
        }

        // Sort words by frequency (highest first)
        List<Map.Entry<String, Integer>> list =
                new ArrayList<>(map.entrySet());

        list.sort((a, b) -> {
            int result = b.getValue().compareTo(a.getValue());

            // If frequency is same, sort alphabetically
            if (result == 0) {
                return a.getKey().compareTo(b.getKey());
            }

            return result;
        });

        // Print result
        System.out.println("Words sorted by frequency:");

        for (Map.Entry<String, Integer> entry : list) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

Output :

```
Console  I/O  AI Agent  1.1s

Words sorted by frequency:
and : 2
easy : 2
is : 2
java : 2
powerful : 1
```

Learning Outcome :

- Learned to use **HashMap** to store word frequencies.
- Learned to split a paragraph into individual words.
- Learned to use a **Comparator/Lambda expression** for sorting.
- Learned to sort data by **frequency in descending order**

Aim – 2 : Unique visitors

From a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.

Code :

Main.java

```java
import java.util.*;

public class UniqueVisitors {
    public static void main(String[] args) {

        List<String> logins = Arrays.asList(
                "Kavya", "Rahul", "Amit", "Kavya",
                "Priya", "Rahul", "Neha", "Amit"
        );

        // HashSet for finding unique users
        HashSet<String> uniqueUsers = new HashSet<>(logins);

        // LinkedHashSet keeps first-seen order
        LinkedHashSet<String> firstSeenUsers =
                new LinkedHashSet<>(logins);

        // Print unique count
        System.out.println("Unique visitors: " + uniqueUsers.size());

        // Print first three distinct names
        System.out.println("First three distinct visitors:");

        int count = 0;

        for (String user : firstSeenUsers) {
            System.out.println(user);
            count++;

            if (count == 3) {
```

dy >_ Terminal

```java
public class UniqueVisitors {
    public static void main(String[] args) {
        List<String> logins = Arrays.asList(
        );

        // HashSet for finding unique users
        HashSet<String> uniqueUsers = new HashSet<>(logins);

        // LinkedHashSet keeps first-seen order
        LinkedHashSet<String> firstSeenUsers =
                new LinkedHashSet<>(logins);

        // Print unique count
        System.out.println("Unique visitors: " + uniqueUsers.size());

        // Print first three distinct names
        System.out.println("First three distinct visitors:");

        int count = 0;

        for (String user : firstSeenUsers) {
            System.out.println(user);
            count++;

            if (count == 3) {
                break;
            }
        }
    }
}
```

Output :

```
Unique visitors: 5
First three distinct visitors:
Kavya
Rahul
Amit
```

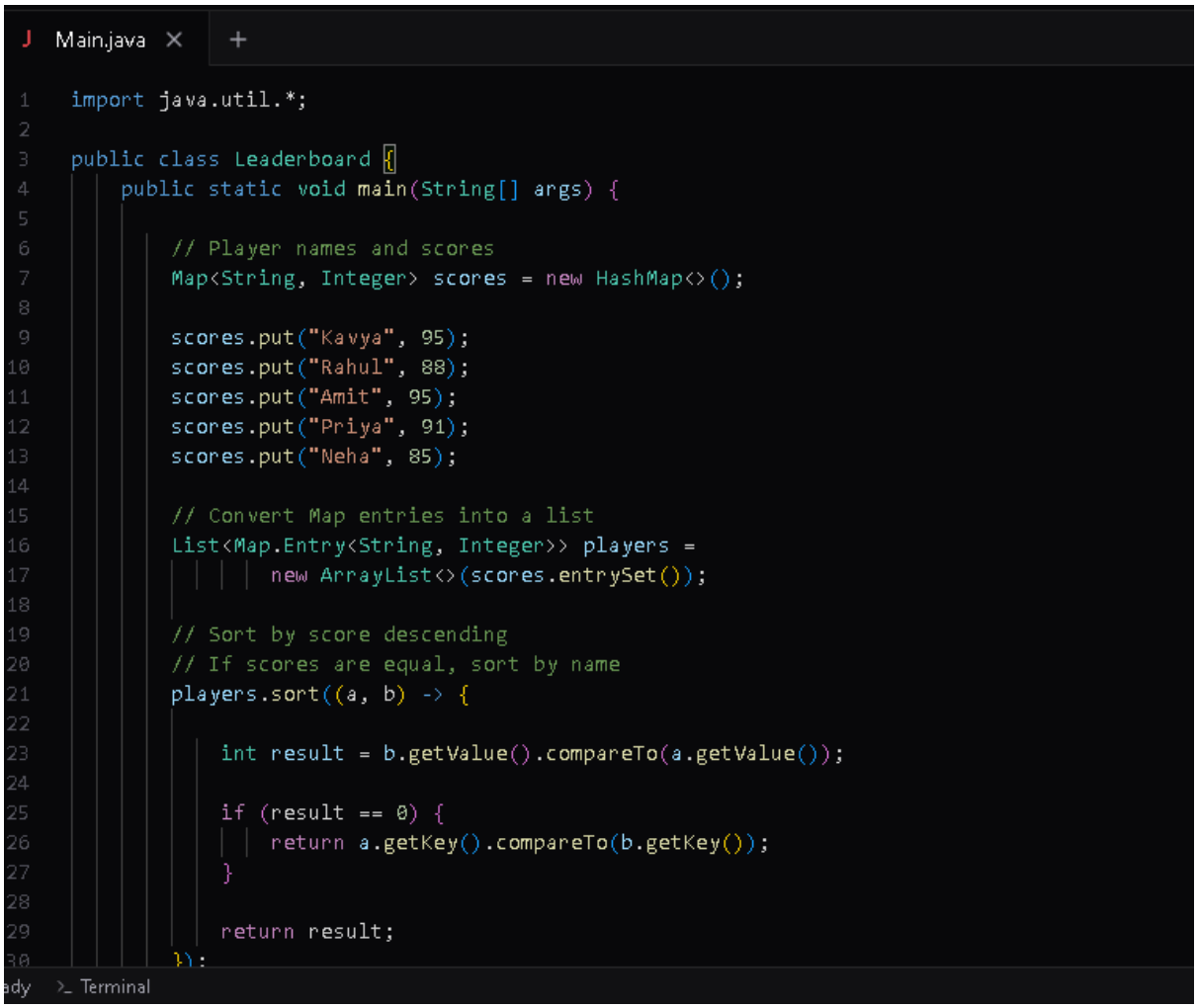
Learning Outcome

- Learned to use **HashSet** to remove duplicate values.
- Learned that a `HashSet` stores only unique elements.
- Learned to use **LinkedHashSet** to maintain insertion/first-seen order.
- Learned how to count unique visitors and display the first three

Aim – 3 : For Advanced Learners – home

Build a leaderboard from player scores held in a Map and print the top-N sorted by score (ties broken by name); choose the data and sorting details.

Code :



```java
public class Leaderboard {
    public static void main(String[] args) {
        // If scores are equal, sort by name
        players.sort((a, b) -> {

            int result = b.getValue().compareTo(a.getValue());

            if (result == 0) {
                return a.getKey().compareTo(b.getKey());
            }

            return result;
        });

        // Top 3 players
        int topN = 3;

        System.out.println("Top " + topN + " Players:");

        for (int i = 0; i < topN && i < players.size(); i++) {
            Map.Entry<String, Integer> player = players.get(i);

            System.out.println(
                    (i + 1) + ". " +
                    player.getKey() + " - " +
                    player.getValue()
            );
        }
    }
}
```

Output :

```
Console   I/O   AI Agent                                  1.1s

Top 3 Players:
1. Amit - 95
2. Kavya - 95
3. Priya - 91
```

Learning Outcome

- Learned to store player scores using a **Map**.
- Learned to convert Map entries into a list for sorting.
- Learned to use a **Comparator** for multiple sorting conditions.
- Learned to sort scores in **descending order**.
- Learned to handle **ties using player names**.
- Learned how to display the **Top-N** players.